

6

Leveraging Data and Services

Managing data is central to the operation of most applications. Learning how to load, maintain, and save that data is an important aspect of WinUI development. Two of the most important aspects of data management are **state management** and the **service locator pattern**. We will be covering these concepts and putting some of them to use in our application.

In this chapter, we will cover the following topics:

- Understanding the WinUI application lifecycle
- Learning to use **SQLite** to store application data
- Learning to use the **object-relational mapper (ORM) Dapper** to quickly map objects in a data service
- Continuing to explore the service locator pattern and implementing it with our data service

By the end of this chapter, you will have a working understanding of the WinUI application lifecycle and will know how to manage data and state in your projects.

Technical requirements

To follow along with the examples in this chapter, the following software is required:

- Windows 10 version 1803 (version 17134) or newer
- Visual Studio 2022 or newer, with the .NET desktop development workload configured for Windows App SDK development

The source code for this chapter is available on GitHub at <https://github.com/PacktPublishing/Learn-WinUI-3-Second-Edition/tree/master/Chapter06>.

Managing application state with app lifecycle events

Before working with data in any application, it is important to understand the application lifecycle for the target application platform. We have touched on these concepts briefly, but now, it's time to take a deeper dive into the Windows application lifecycle for **WinUI on Desktop** applications.

Exploring Windows application lifecycle events

WinUI on desktop applications has a slightly different set of lifecycle events than other desktop .NET applications. WPF and **Windows Forms (WinForms)** applications are either running or they're not. There are several events that occur while launching and shutting down WPF and WinForms applications:

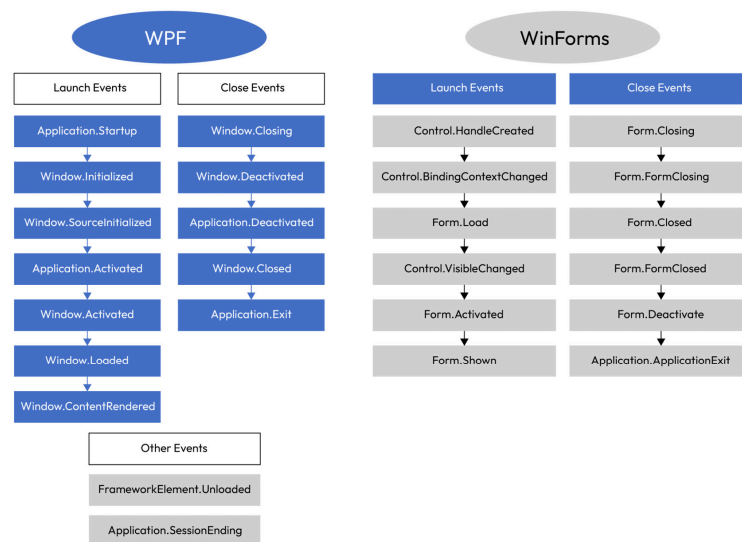


Figure 6.1 – WPF and WinForms application lifecycle events

NOTE

We won't go into the details here, as our primary focus is building WinUI 3 applications. However, for the two WPF events that fall outside of the launch and close, their sequences are as follows:

1. **FrameworkElement.Unloaded**: This event fires when an element is removed from the WPF visual tree. It does not fire during application shutdown.
2. **Application.SessionEnding**: This event fires when the current Windows user logs off or shuts down Windows. In the event handler, you can request that Windows cancels the process by setting the **SessionEndingCancelEventArgs.Cancel** property to **true**.

Lifecycle events of WinUI applications

Let's talk about the WinUI lifecycle. Lifecycle events give you a chance to initialize any data and state when your application starts execution, and this allows you to clean up or save the state when the application is closed. In UWP applications, you also had the ability to handle events when the application was suspended or resumed because of a user or operating system action. WinUI, like other .NET desktop applications, doesn't have this ability.

In the **Application** and **Window** classes, there are only a couple of events that can be handled. Every **Application** class overrides the **OnLaunched**

method. This method will be invoked exactly once, when the application is launched either by a user or the operating system. We've already worked with the **OnLaunched** method in our sample application. It's where **MainWindow** is created and where we added a call to the method to configure our IOC container. In a new WinUI application, the **OnLaunched** method will look like this:

```
protected override void OnLaunched(Microsoft.UI.Xaml.LaunchActivatedEventArgs args)
{
    m_window = new MainWindow();
    m_window.Activate();
}
```

The **Application** and **Window** classes only inherit from the .NET **Object** class, so there are no inherited events to leverage either. This is not the case with the **Page** class, which we will discuss shortly. First, we're going to discuss **Window**, which has a couple of events we can leverage when managing an app's lifecycle.

The **Window** class in WinUI does not have a **Loaded** event, which is used in a WPF **Window** class to indicate that the window and its contents are loaded and available for interaction. The **Window.Activated** event can be used in place of a **Loaded** event, but the **Activated** event fires every time the window receives focus. If this is your only option, you will need to add a flag to check whether it's the first time that **Activated** has fired.

The other lifecycle event in **Window** is the **Closed** event, which fires when the window has closed. If it is the last remaining window, usually **MainWindow**, the application will end after the window closes. This is where you should save any application data and state information.

This is the extent of the application lifecycle events provided out of the box by the **Application** and **Window** classes. However, you can tap into some other events provided by **FrameworkElement**, which is a base class of **Page** and all the other controls in WinUI.

Additional lifecycle events with FrameworkElement objects

Every control in WinUI inherits from **Control**, which inherits from **FrameworkElement**. Even the **Page** control, which we added to the **My Media Collection** app to provide navigation to an item details view, inherits from **Control**, through its inheritance from **UserControl**.

The **FrameworkElement** class provides three useful events that developers can leverage for the application lifecycle:

- **Loading**: This event occurs when the loading process has started. This event can be leveraged to start fetching and processing data from services or other sources. You could also start loading data sooner, in the constructor of the current **Window** or **Page**, or even in **Application.OnLoaded**.

- **Loaded:** The **Loaded** event is invoked when the current element and all its children are loaded and ready for interaction. Do not try to manipulate these elements before they are loaded, or the app will raise a runtime exception.
- **Unloaded:** This event is fired when the current element has been unloaded and removed from the **visual tree**. If you are handling this event for **Page**, you can use this to clean up resources or save any state for the page.

NOTE

We haven't discussed the WinUI visual tree up to this point. The concept of trees, physical and logical, in WinUI is the same as in other XAML frameworks. We will discuss the physical tree and logical tree in more detail when we discuss debugging WinUI applications in [Chapter 11, Debugging WinUI Apps with Visual Studio](#). To learn more now, Microsoft Learn has a great WPF article about trees:

<https://learn.microsoft.com/dotnet/desktop/wpf/advanced/trees-in-wpf>.

You can handle the **Loaded** event for any control in your current view, whether it's **Window** or **Page**, but always keep performance in mind. The **Loaded** event for the topmost **FrameworkElement** in the visual tree will not be fired until those of all its children have been completed. Network and filesystem operations can be expensive, so try to minimize and consolidate the calls to load the data needed to present your view whenever possible.

NOTE

For more information about handling lifecycle events, read the following Microsoft Learn page:

<https://learn.microsoft.com/windows/apps/windows-app-sdk/applifecycle/applifecycle>.

Now that you have a solid understanding of WinUI's lifecycle, let's start working with some real data that will need to be persisted between user sessions.

Creating a SQLite data store

Until this point, the **My Media Collection** project has only worked with data stored inside in-memory collections. This means that every time the application is closed, all the user's data is lost. It has also meant calling a method to populate all the lists with hardcoded seed data each time the application is launched.

In the previous chapter, we took the first step in creating a maintainable data service for the application. By creating a data service class that implements **IDataService**, no changes will be required in the **ViewModel** classes when we start loading data from a database. This section will focus on creating a new **SQLiteDataService** class so that we can use **SQLite**

for data access. The starting code for this chapter can be found on GitHub at <https://github.com/PacktPublishing/Learn-WinUI-3-Second-Edition/tree/master/Chapter06/Start>.

What is SQLite?

SQLite (found at <https://sqlite.org/>) is a SQL-based database that is frequently used by mobile apps and simple desktop applications. It is a popular choice because it is small, fast, and self-contained in a single file. There are SQLite libraries available for virtually every platform. We will use Microsoft's **Microsoft.Data.Sqlite** ADO.NET provider for SQLite.

NOTE

For more information about Microsoft's SQLite provider, you can read <https://learn.microsoft.com/dotnet/standard/data/sqlite/>. To learn more about using SQLite with WinUI projects, check out this Microsoft Learn article: <https://learn.microsoft.com/windows/apps/develop/data-access/sqlite-data-access>.

Adding SQLite as a data service

Follow these steps:

1. Start by adding the **Microsoft.Data.Sqlite** NuGet package to the **MyMediaCollection** project by opening **Package Manager Console** from **View | Other Windows | Package Manager Console** and running the following command. Also, make sure the **MyMediaCollection** project is selected in the **Package Manager Console** window's project dropdown before running this command:

```
Install-Package Microsoft.Data.Sqlite
```

Running this command is equivalent to finding and adding the package from the **NuGet Package Manager** window.

2. When the process is complete, create a new class named **SqliteDataService** in the **Services** folder, and copy the contents of the **DataService** class into it as a starting point.
3. Now, open **SqliteDataService.cs** and start by adding four **using** statements to the top of the file:

```
using Microsoft.Data.Sqlite;  
using System.IO;  
using System.Threading.Tasks;  
using Windows.Storage;
```

The **System.IO** and **Windows.Storage** namespaces will be used when we initialize the SQLite database file, and we'll need the **System.Threading.Tasks** namespace imported to work with some **async** tasks.

4. Next, add a new constant to the class to hold the database's filename:

```
private const string DbName = "mediaCollectionData.db";
```

5. Now, let's create a private method to create or open the database file, create a **SqlConnection** class for the database, open it, and return it to the caller. This method can be used throughout the class whenever a new database connection is needed. The database file will be created in the user's **LocalFolder**, which means the application's data will be saved with the user's local Windows profile data:

```
private async Task<SqlConnection> GetOpenConnectionAsync()
{
    await ApplicationData.Current.LocalFolder.CreateFileAsync(DbName, CreationCollisionOption.O
string dbPath = Path.Combine(ApplicationData.Current.LocalFolder.Path, DbName);
var cn = new SqlConnection($"Filename={dbPath}");
cn.Open();
return cn;
}
```

Note that we have declared this method as **async** and that it uses the **await** keyword when opening or creating the file. It is a good practice to use **async/await** when using external resources such as files, network connections, or databases to keep your application responsive.

NOTE

To find out more about **async/await** with C# and .NET, Microsoft Learn has a great article to get you started:

<https://learn.microsoft.com/dotnet/csharp/asynchronous-programming/>.

6. Next, create two methods that will create the **MediaItems** and **Mediums** tables in the database. These will be called each time the app launches, but the SQL code only creates the tables if they do not exist. The **SqlCommand** object accepts the **tableCommand** query string and **SqlConnection**. It has several methods it can use to execute the command, depending on whether any data is expected to be returned by the query. In our case, no return values are expected, so **ExecuteNonQueryAsync** is the best of the **async** options for these two methods:

```
private async Task CreateMediumTableAsync(SqliteConnection db)
{
    string tableCommand = @"CREATE TABLE IF NOT
        EXISTS Mediums (Id INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
        Name NVARCHAR(30) NOT NULL,
        MediumType INTEGER NOT NULL)";
    using var createTable = new SqlCommand(tableCommand, db);
    await createTable.ExecuteNonQueryAsync();
}
private async Task CreateMediaItemTableAsync(SqliteConnection db)
{
    string tableCommand = @"CREATE TABLE IF NOT
        EXISTS MediaItems (Id INTEGER PRIMARY KEY AUTOINCREMENT,
        Name NVARCHAR(1000) NOT NULL,
        ItemType INTEGER NOT NULL,
        MediumId INTEGER NOT NULL,
        LocationType INTEGER,
        CONSTRAINT fk_mediums
```

```

        FOREIGN KEY(MediumId)
        REFERENCES Mediums(Id))";
        using var createTable = new SqliteCommand(tableCommand, db);
        await createTable.ExecuteNonQueryAsync();
    }

```

7. Now, create a method for the **Mediums** table to insert a row into the table:

```

private async Task InsertMediumAsync(SqliteConnection db, Medium medium)
{
    using var insertCommand = new SqliteCommand
    {
        Connection = db,
        CommandText = "INSERT INTO Mediums VALUES (NULL, @Name, @MediumType);";
    };
    insertCommand.Parameters.AddWithValue("@Name", medium.Name);
    insertCommand.Parameters.AddWithValue("@MediumType", (int)medium.MediaType);
    await insertCommand.ExecuteNonQueryAsync();
}

```

8. Now, we need another method that will read all the rows from the **Mediums** table:

```

private async Task<IList<Medium>> GetAllMediumsAsync(SqliteConnection db)
{
    IList<Medium> mediums = new List<Medium>();
    using var selectCommand = new SqliteCommand("SELECT Id, Name, MediumType FROM Mediums", db)
    using SqlDataReader query = await selectCommand.ExecuteReaderAsync();
    while (query.Read())
    {
        var medium = new Medium
        {
            Id = query.GetInt32(0),
            Name = query.GetString(1),
            MediaType = (ItemType)query.GetInt32(2)
        };
        mediums.Add(medium);
    }
    return mediums;
}

```

There's a bit of code needed for these two simple operations. The insert method needs to add parameters for each property to be saved in the table, and the select method uses a **while** loop to add each table's record to the collection. Let's see whether we can simplify this in the next section.

Before we implement the remaining methods for the **Create, Read, Update, Delete (CRUD)** operations, a new library must be added to the project to simplify the data access code we will write.

Leveraging a Micro ORM to simplify data access

As you saw in the previous section, writing data access code for even the simplest application can take some time.

ORMs, such as **Entity Framework Core (EF Core)**, can greatly simplify and reduce the code required, but they can be overkill for a small app with just a handful of tables. In this chapter, we'll look at a **Micro ORM**. Micro ORMs are lightweight frameworks that handle mapping data between objects and data queries.

NOTE

EF Core is a popular ORM for .NET developers. If you want to learn more about how to use EF Core with your projects, you can view the Packt video [Entity Framework Core – a Full Tour at](https://www.packtpub.com/product/entity-framework-core-a-full-tour-net-5-and-up-video/9781803242231)

<https://www.packtpub.com/product/entity-framework-core-a-full-tour-net-5-and-up-video/9781803242231>.

The framework we will be using for data access in our project, **Dapper**, is an open source .NET Micro ORM that was created by the developers at **Stack Overflow**. You can learn more about Dapper at <https://dapperlib.github.io/Dapper/> and get the package on NuGet: <https://www.nuget.org/packages/Dapper>.

Dapper is popular within the .NET community. While it doesn't offer some of the features of EF Core, such as model generation or entity change tracking, it does make it very easy to write a fast, slim data layer. When you add the **Dapper.Contrib** library (<https://www.nuget.org/packages/Dapper.Contrib>) into the mix, it is even easier to write the CRUD methods needed for your applications.

Adding Dapper to the project

Let's dive right into it:

1. Start by adding **Dapper** and **Dapper.Contrib** to the **MyMediaCollection** project. Open the **Package Manager Console** window again and add the two packages to your project:

```
Install-Package Dapper
Install-Package Dapper.Contrib
```

2. Now, revisit the **InsertMediaAsync** method. If we use the **QueryAsync** method provided by Dapper, we can reduce the code from our original method to this:

```
private async Task InsertMediumAsync(SqliteConnection db, Medium medium)
{
    var newIds = await db.QueryAsync<long>(
        $"INSERT INTO Mediums
        ({nameof(medium.Name)}, MediumType)
        VALUES
        (@{nameof(medium.Name)}, @{nameof(medium.MediaType)});
        SELECT last_insert_rowid()", medium);
    medium.Id = (int)newIds.First();
}
```

The code we wrote to set the values of the query parameters is now gone. Dapper maps them for us from the **medium** object, which is

passed into its **QueryAsync** method. You must ensure that the parameter names in the SQLite query match the property names on our object for Dapper's automatic mapping to work.

3. As a bonus, we can also get the generated ID back from the **QueryAsync** call by adding the following SQLite code, which will return it after the **INSERT** operation completes:

```
SELECT last_insert_rowid();
```

4. Next, update the code for **GetAllMediumsAsync** to use Dapper:

```
private async Task<IList<Medium>> GetAllMediumsAsync(SqliteConnection db)
{
    var mediums =
        await db.QueryAsync<Medium>(@"SELECT Id,
                                    Name,
                                    MediumType AS MediaType
                                    FROM Mediums");

    return mediums.ToList();
}
```

We've gone from 14 lines of code to only 2. Note, in the highlighted part of the query, how we use an alias of **MediaType** for the **MediumType** field. This is an easy way to map data to an object property that doesn't match the database field name, by simply renaming the field that's returned as part of the SQL select statement. Dapper has also helped us by directly returning a list of our **Medium** objects, instead of us having to use a **while** loop to iterate over the result set.

5. Next, create a query that will get all the media items to populate the main **ListView** control. This query is a little more complex because we join two tables, **MediaItems** and **Mediums**, on **MediumId** and return the data to be mapped to two corresponding objects, **item** and **medium**. These types are indicated by the first two generic types provided to the **QueryAsync** method. To perform this mapping, we give Dapper a lambda expression that directs it to set **medium** as the **MediumInfo** property of the item for each row that's returned from the query. The type of the returned object is defined by the third generic type provided to the **QueryAsync** method. The remaining parameters will be mapped automatically by Dapper based on their property names:

```
private async Task<List<MediaItem>> GetAllMediaItemsAsync(SqliteConnection db)
{
    var itemsResult = await db.QueryAsync<MediaItem, Medium, MediaItem>
        (
            @"SELECT
                [MediaItems].[Id],
                [MediaItems].[Name],
                [MediaItems].[ItemType] AS MediaType,
                [MediaItems].[LocationType] AS Location,
                [Mediums].[Id],
                [Mediums].[Name],
                [Mediums].[MediumType] AS MediaType
            FROM
                [MediaItems]
            JOIN
                [Mediums]
```

```

        ON
        [Mediums].[Id] = [MediaItems].[MediumId]",
        (item, medium) =>
        {
            item.MediumInfo = medium;
            return item;
        }
    );
    return itemsResult.ToList();
}

```

6. Next, add the code to create the insert and update methods for our media items:

```

private async Task<int> InsertMediaItemAsync(SqliteConnection db, MediaItem item)
{
    var newIds = await db.QueryAsync<long>(
        @"INSERT INTO MediaItems
        (Name, ItemType, MediumId, LocationType)
        VALUES
        (@Name, @MediaType, @MediumId, @Location);
        SELECT last_insert_rowid()", item);
    (int)newIds.First();
}

private async Task UpdateMediaItemAsync(SqliteConnection db, MediaItem item)
{
    await db.QueryAsync(
        @"UPDATE MediaItems
        SET Name = @Name,
            ItemType = @MediaType,
            MediumId = @MediumId,
            LocationType = @Location
        WHERE Id = @Id;", item);
}

```

The code in **InsertMediaItemAsync** should look familiar. It's very similar to what we did when we inserted data into the **Mediums** table. The code to update a row in **MediaItems** is technically only one line now, thanks to **Dapper**.

7. There's one new read-only property that's been added to the **MediaItem** object in our model. This property allows **Dapper** to map **MediumId** to the **MediaItems** table:

```
public int MediumId => MediumInfo.Id;
```

8. Now, add the **Computed** attribute to the **MediaItem.MediumInfo** property. This tells **Dapper** to ignore the property when we attempt to insert or update rows in the database. We only need to have **MediumId** saved. Users are unable to make changes to the rows in the **Mediums** table:

```

[Computed]
public Medium MediumInfo { get; set; }

```

9. Finally, let's create a method that will delete items from the **MediaItems** table. This code is a little different, thanks to **Dapper.Contrib**. We don't need to write any parameterized SQL in the code because **Dapper.Contrib** has a **DeleteAsync** method that gener-

ates the code to delete from **MediaItems**, based on the **Id** property of the **MediaItem** class provided:

```
private async Task DeleteMediaItemAsync(SqliteConnection db, int id)
{
    await db.DeleteAsync<MediaItem>(new MediaItem { Id = id });
}
```

To make this work, you must decorate the primary key properties of your model classes with **Key** attributes:

```
public class MediaItem
{
    [Key]
    public int Id { get; set; }
    ...
}
```

Make sure that every model class that uses one of the **Dapper.Contrib** attributes adds a **using** statement for **Dapper.Contrib.Extensions**.

Before we update all the public CRUD methods of the **SqliteDataService** class to call these private methods, we will complete the code that initializes the service when the application launches.

Updating the data service's initialization

Let's get started:

1. First, create a version of the **DataService.PopulateMediums** method in **Sqlite DataService**, make it **async**, and rename it **PopulateMediumsAsync**. Update this method so that it fetches the data from SQLite. The method will also create any required data if this is the first time the application has been launched for the current user:

```
private async Task PopulateMediumsAsync(SqliteConnection db)
{
    _mediums = await GetAllMediumsAsync(db);
    if (_mediums.Count == 0)
    {
        var cd = new Medium { Id = 1, MediaType = ItemType.Music, Name = "CD" };
        var vinyl = new Medium { Id = 2, MediaType = ItemType.Music, Name = "Vinyl" };
        var hardcover = new Medium { Id = 3, MediaType = ItemType.Book, Name = "Hardcover" };
        var paperback = new Medium { Id = 4, MediaType = ItemType.Book, Name = "Paperback" };
        var dvd = new Medium { Id = 5, MediaType = ItemType.Video, Name = "DVD" };
        var bluRay = new Medium { Id = 6, MediaType = ItemType.Video, Name = "Blu Ray" };
        var mediums = new List<Medium>
        {
            cd, vinyl, hardcover, paperback, dvd, bluRay
        };
        foreach (var medium in mediums)
        {
            await InsertMediumAsync(db, medium);
        }
        _mediums = await GetAllMediumsAsync(db);
    }
}
```

2. Second, remove **PopulateItems** from **SqliteDataService**, **DataService**, and **IDataService**. It will not be needed because we now persist all data between sessions. You can also remove the **_items** private variable.
3. Now, take the code from the **SqliteDataService** constructor, move it to a new public method named **InitializeDataAsync**, and update the code so that it uses the new private initialization methods. Don't forget to remove the call to populate the items collection. The **SqlConnection** object should always be part of a **using** block to ensure that the connection is closed and the object is disposed of:

```
public async Task InitializeDataAsync()
{
    using (var db = await GetOpenConnectionAsync())
    {
        await CreateMediumTableAsync(db);
        await CreateMediaItemTableAsync(db);
        SelectedItemId = -1;
        PopulateItemTypes();
        await PopulateMediumsAsync(db);
        PopulateLocationTypes();
    }
}
```

4. This new initialization method will need to be added to **IDataService** to make it available to objects that resolve the service through our DI container. If you keep the original **DataService** class in your project, you will need to add an implementation of **InitializeDataAsync** so that the project will compile:

```
public interface IDataService
{
    Task InitializeDataAsync();
    ...
}
```

5. After changing the location of the code that initializes **SqliteDataService**, the **RegisterComponents** method in **App.xaml.cs** will need to be updated to use the new **SqliteDataService** and call **InitializeDataAsync**. While we're at it, rename the method to reflect its new async status:

```
private async Task RegisterComponentsAsync(Frame rootFrame)
{
    var navigationService = new NavigationService(rootFrame);
    navigationService.Configure(nameof(MainPage), typeof(MainPage));
    navigationService.Configure(nameof(ItemDetailsPage), typeof(ItemDetailsPage));
    var dataService = new SqliteDataService();
    await dataService.InitializeDataAsync();
    HostContainer = Host.CreateDefaultBuilder()
        .ConfigureServices(services =>
        {
            services.AddSingleton<INavigationService>(navigationService);
            services.AddSingleton<IDataService>(dataService);
            services.AddTransient<MainViewModel>();
            services.AddTransient<ItemDetailsViewModel>();
        })
        .Build();
}
```

```
    }).Build();
}
```

Don't forget to update **OnLaunched** so that it's **async** and await the call to the renamed **RegisterComponentsAsync**.

Now that the application initializes the data service when it launches, it's time to update the public CRUD methods so they use the async private methods we created to fetch data from SQLite.

Retrieving data via services

Let's start retrieving and saving SQLite data with our service methods. It will only be necessary to update the create, update, and delete operations. All the media items are stored in **List<MediaItem>** in **DataService**, so the public methods used to retrieve items can remain as they were in the previous chapter. Let's get started:

1. Start by updating the create, update, and delete methods for the media items in **SqliteDataService.cs**. Each of these will get an open connection to the database from **GetOpenConnectionAsync** and call its corresponding private method asynchronously:

```
public async Task<int> AddItemAsync(MediaItem item)
{
    using var db = await GetOpenConnectionAsync();
    return await InsertMediaItemAsync(db, item);
}
public async Task UpdateItemAsync(MediaItem item)
{
    using var db = await GetOpenConnectionAsync();
    await UpdateMediaItemAsync(db, item);
}
public async Task DeleteItemAsync(MediaItem item)
{
    using var db = await GetOpenConnectionAsync();
    await DeleteMediaItemAsync(db, item.Id);
}
```

2. Update the public methods that fetch items to be async:

```
public async Task<MediaItem> GetItemAsync(int id)
{
    IList<MediaItem> mediaItems;
    using var db = await GetOpenConnectionAsync();
    mediaItems = await GetAllMediaItemsAsync(db);
    // Filter the list to get the item for our Id.
    return mediaItems.FirstOrDefault(i => i.Id == id);
}
public async Task<IList<MediaItem>> GetItemsAsync()
{
    using var db = await GetOpenConnectionAsync();
    return await GetAllMediaItemsAsync(db);
}
```

NOTE

*If you need to do a lot of filtering when querying data, Entity Framework is a more robust ORM that can provide more extensive options. SQLite is best suited for simpler applications. Note that, in the preceding code, **GetItemAsync** queries all the items and then filters to the item that matches the provided ID by using a lambda expression.*

3. The method names have been updated to include **Async**, all uses of the **_items** collection have been removed, and each method has been changed to return **Task**. So, update the **IDataService** interface members to reflect the same changes. Also, either remove **DataService** from the project or update its methods to also be async. It's best to try and anticipate that data access methods will need to be async when you set out, thus preventing breaking changes to your interfaces:

```
Task<int> AddItemAsync(MediaItem item);
Task UpdateItemAsync(MediaItem item);
Task DeleteItemAsync(MediaItem item);
Task<IList<MediaItem>> GetItemsAsync();
Task<MediaItem> GetItemAsync(int id);
```

4. In **MainViewModel.cs**, the **Delete** method will be updated to use async/await with its data service call. Don't forget to rename it **DeleteAsync** to follow best practices when naming async methods. You will also need to add a **using** statement to the file for the **System.Threading.Tasks** namespace:

```
private async Task DeleteAsync()
{
    await _dataService.DeleteItemAsync(SelectedMediaItem);
    Items.Remove(SelectedMediaItem);
    allItems.Remove(SelectedMediaItem);
}
```

5. Update the **PopulateData** method so that it's named **PopulateDataAsync**, and use the async method of getting items:

```
public async Task PopulateDataAsync()
{
    items.Clear();
    foreach(var item in await _dataService.GetItemsAsync())
    {
        items.Add(item);
    }
    allItems = new ObservableCollection<MediaItem>(Items);
    mediums = new ObservableCollection<string>
    {
        AllMediums
    };
    foreach(var itemType in _dataService.GetItemTypes())
    {
        mediums.Add(itemType.ToString());
    }
    selectedMedium = Mediums[0];
}
```

6. Now, you will have to update the **MainViewModel** constructor to call to

PopulateDataAsync at the end of the constructor:

```
public MainViewModel(INavigationService navigationService, IDataService dataService)
{
    _navigationService = navigationService;
    _dataService = dataService;
    PopulateDataAsync();
}
```

7. Some similar changes will be needed in **ItemDetailsViewModel**.

Update the **Save** method so that it's async and awaits the data service calls to **AddItemAsync**, **GetItemAsync**, and **UpdateItemAsync**. Don't forget to rename **Save** to **SaveAsync** and add a **using** statement for the **System.Threading.Tasks** namespace:

```
private async Task SaveAsync()
{
    MediaItem item;
    if (_itemId > 0)
    {
        item = await _dataService.GetItemAsync(_itemId);
        item.Name = ItemName;
        item.Location = (LocationType)Enum.Parse(typeof(LocationType), SelectedLocation);
        item.MediaType = (ItemType)Enum.Parse(typeof(ItemType), SelectedItemType);
        item.MediumInfo = _dataService.GetMedium(SelectedMedium);
        await _dataService.UpdateItemAsync(item);
    }
    else
    {
        item = new MediaItem
        {
            Name = ItemName,
            Location = (LocationType)Enum.Parse(typeof(LocationType), SelectedLocation),
            MediaType = (ItemType)Enum.Parse(typeof(ItemType), SelectedItemType),
            MediumInfo = _dataService.GetMedium(SelectedMedium)
        };
        await _dataService.AddItemAsync(item);
    }
}
```

8. Next, update the **SaveItemAndReturn** and **SaveAndContinue** methods so that they also use async/await:

```
private async Task SaveItemAndReturnAsync()
{
    await SaveItemAsync();
    _navigationService.GoBack();
}
private async Task SaveItemAndContinueAsync()
{
    await SaveItemAsync();
    _dataService.SelectedItemId = 0;
    _itemId = 0;
    ItemName = "";
    SelectedMedium = null;
    SelectedLocation = null;
    SelectedItemType = null;
}
```

```
IsDirty = false;
}
```

9. Finally, update **ItemDetailsViewModel.xaml** so that the save buttons use the async methods when they bind their **Click** methods:

```
<SplitButton x:Name="SaveButton"
    Content="Save and Return"
    Margin="8,8,0,8"
    Click="{x:Bind ViewModel.SaveItemAndReturnAsync}"
    IsEnabled="{x:Bind ViewModel.IsDirty, Mode=OneWay}">
    ...
<Button Content="Save and Create New"
    Click="{x:Bind ViewModel.SaveItemAndContinueAsync}"
    IsEnabled="{x:Bind ViewModel.IsDirty, Mode=OneWay}"
    Background="Transparent"/>
<Button Content="Save and Return"
    Click="{x:Bind ViewModel.SaveItemAndReturnAsync}"
    IsEnabled="{x:Bind ViewModel.IsDirty, Mode=OneWay}"
    Background="Transparent"/>
```

NOTE

*If you have any issues with the **Media Type ComboBox** not populating on the main page after adding or editing a media item, update **Mode** of its **ItemsSource** data binding to **OneWay**. The completed source code in [GitHub](#) has been updated to reflect this change.*

That's it. Run the application and see how it works. Since we're no longer creating any dummy data for the media items list, the media collection in **ListView** will be empty when the app launches for the first time:

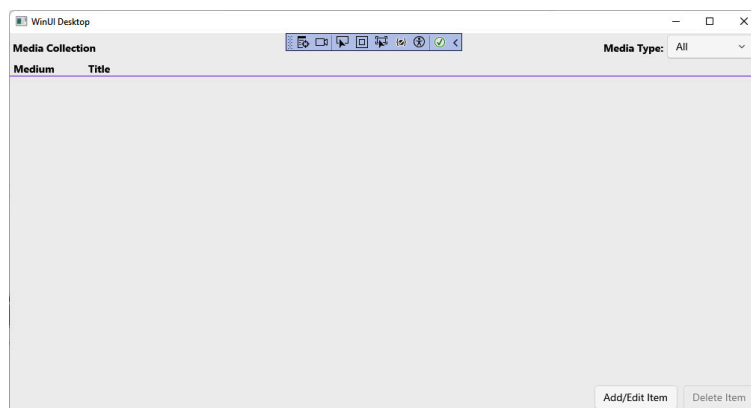


Figure 6.2 – Launching with a database for the first time

Try adding, updating, and removing some items. Then, close the application and run it again. You should see the same items on the list that were there when you closed it:

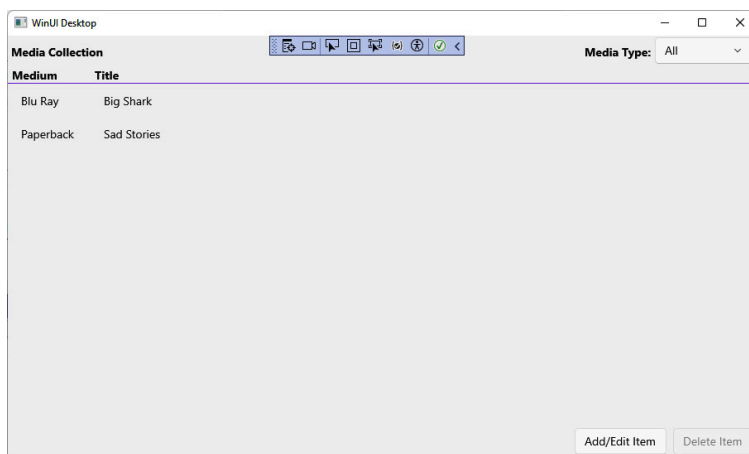


Figure 6.3 – Relaunching with saved data

Users can now retain their saved data. If you would like to browse your SQLite data outside your app, there are tools you can use to connect to a local **db** and inspect it. One of them is **DB Browser for SQLite**. Covering this tool is beyond the scope of this book, but you can explore it for yourself at <https://sqlitebrowser.org/>.

Let's wrap up and review what we've learned about working with data in a WinUI application.

Summary

We have covered a lot of important material in this chapter. You learned how to read and write data to a local SQLite database. Then, you learned how to simplify your data access code by leveraging Dapper, an ORM for .NET developers. Using an ORM will save you time creating boilerplate mapping code in your data access layers for WinUI projects (or any other .NET projects). All this data access code was made async to keep the UI responsive for the user.

In the next chapter, we will learn how to create a beautiful **Fluent UI** with Microsoft's Fluent UI design principles.

Questions

1. When will Windows put a WinUI 3 app into a suspended state?
2. When should you save the app state to ensure it is not lost if an application is closed?
3. What is the event you can handle on a **Page** class to perform some logic when every element on a page has finished loading?
4. What is a Micro ORM?
5. What is the name of the Dapper package that adds CRUD helpers such as **Delete** and **DeleteAsync**?
6. What is one of the powerful features of some more full-featured ORMs, such as Entity Framework?
7. What is the name of a tool that can be used to inspect data in a SQLite database?

